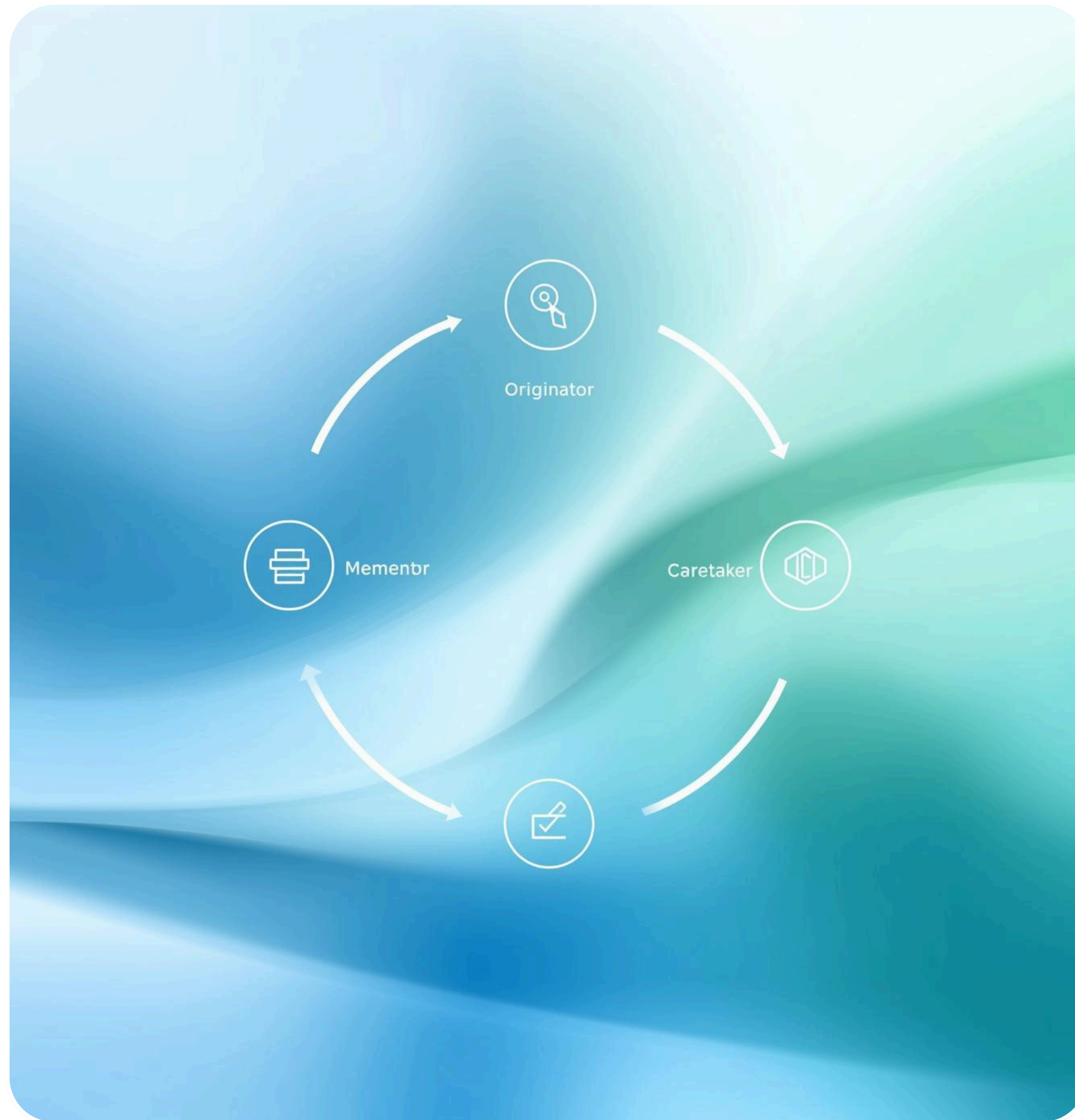




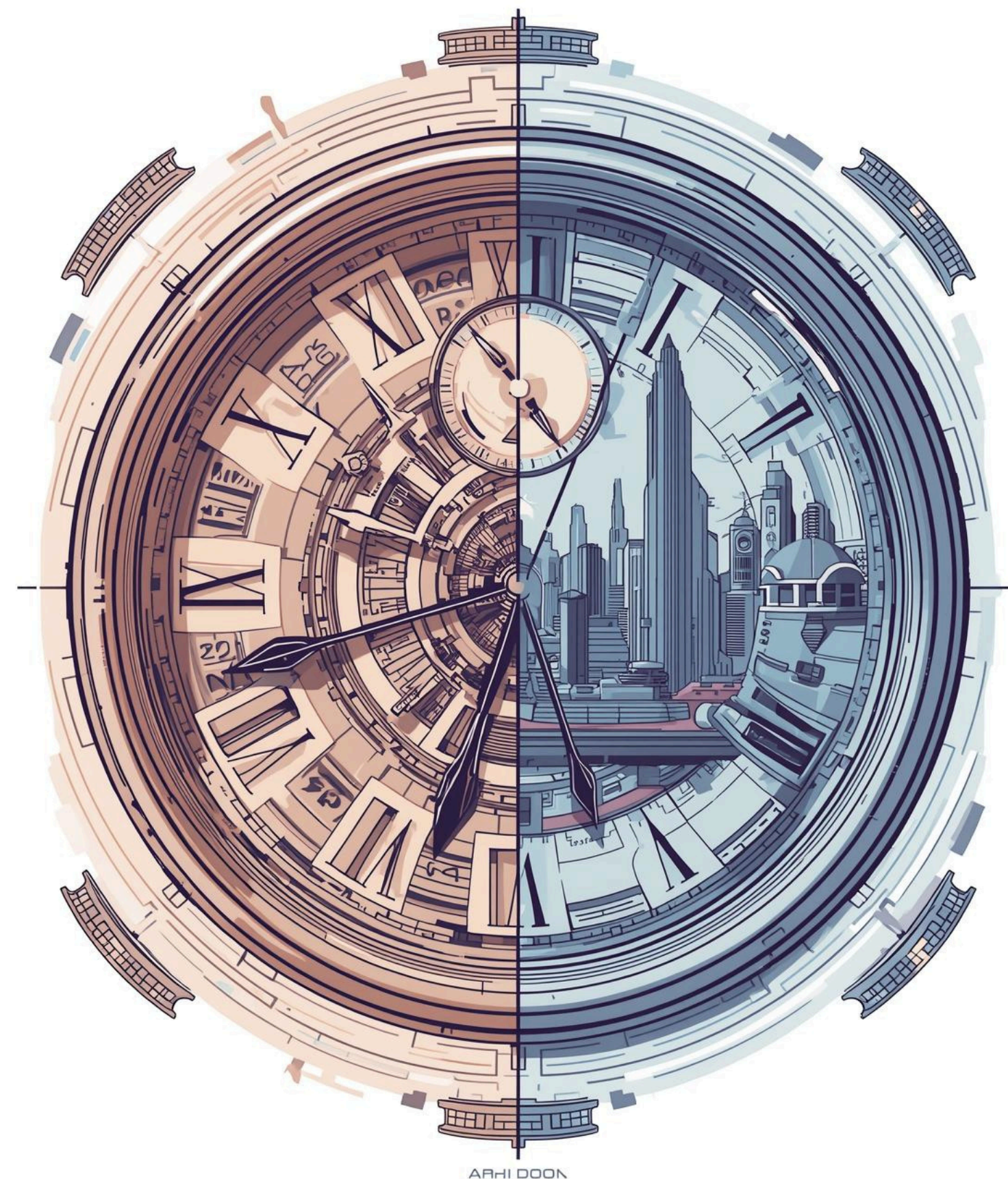
Padrão de Projeto Memento

Breno Augusto de Oliveira SC3018059

Fernando Candido Rodrigues SC3018237

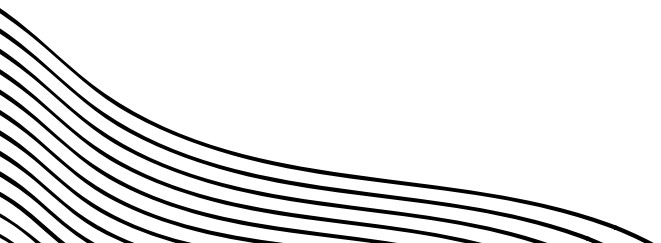
Definição

Memento é um padrão de projeto **comportamental** seu objetivo é **salvar e restaurar** o estado interno de um objeto sem violar o **princípio do encapsulamento**

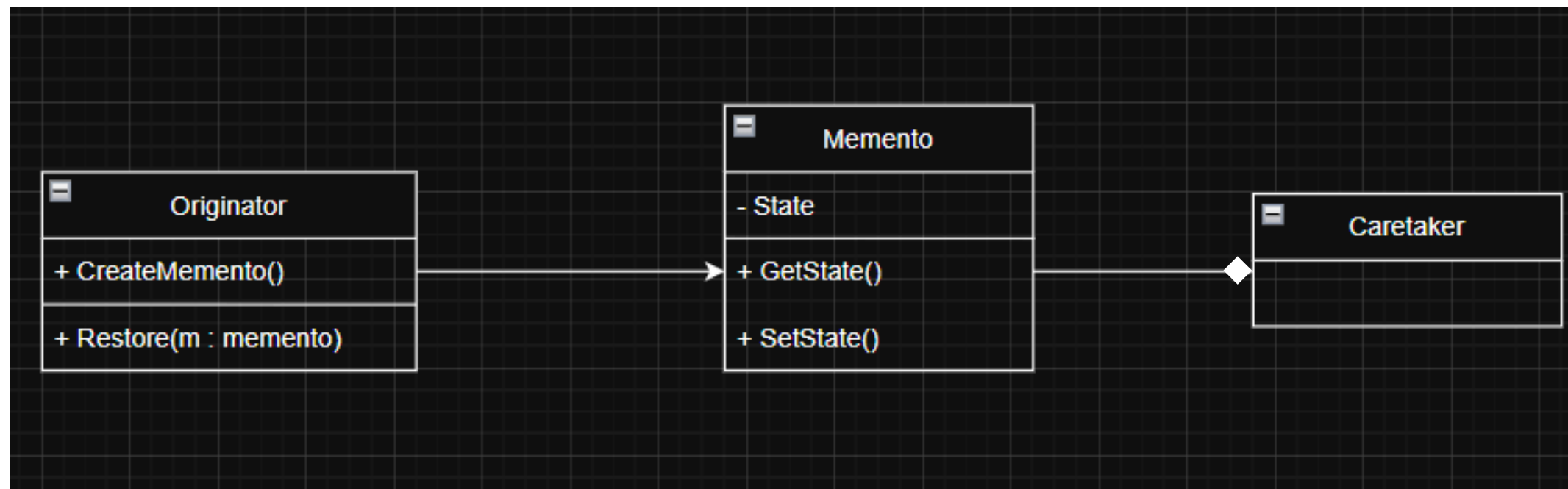


Motivação

Como **restaurar** ou **desfazer** algo em um sistema orientado a objetos sem **quebrar** o **encapsulamento**?



Estrutura do Padrão



1. **Originator** (Classe Original)

- `CreateMemento()` → cria um memento
- `Restore(memento)` → restaura a partir de um memento

2. **Memento**

Objeto que armazena o estado interno do Originator.
Só o Originator deve ter acesso aos seus detalhes.

3. **Caretaker (Zelador)**

Armazena os mementos, mas não pode inspecionar seu conteúdo.

Exemplo de Implementação

Imagine um programa que permite desenhar figuras geométricas (como círculos e quadrados) em uma tela.

O usuário pode **mover**, **redimensionar** ou **mudar** a cor das figuras.

No entanto ele também quer ter a opção de **desfazer** a última modificação feita em uma figura.

Repositório : <https://github.com/RodrigueFernando/Padrao-Memento/tree/main>

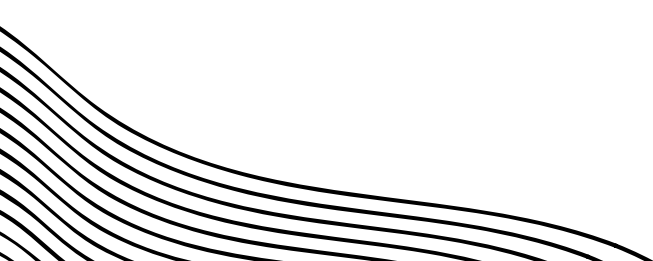
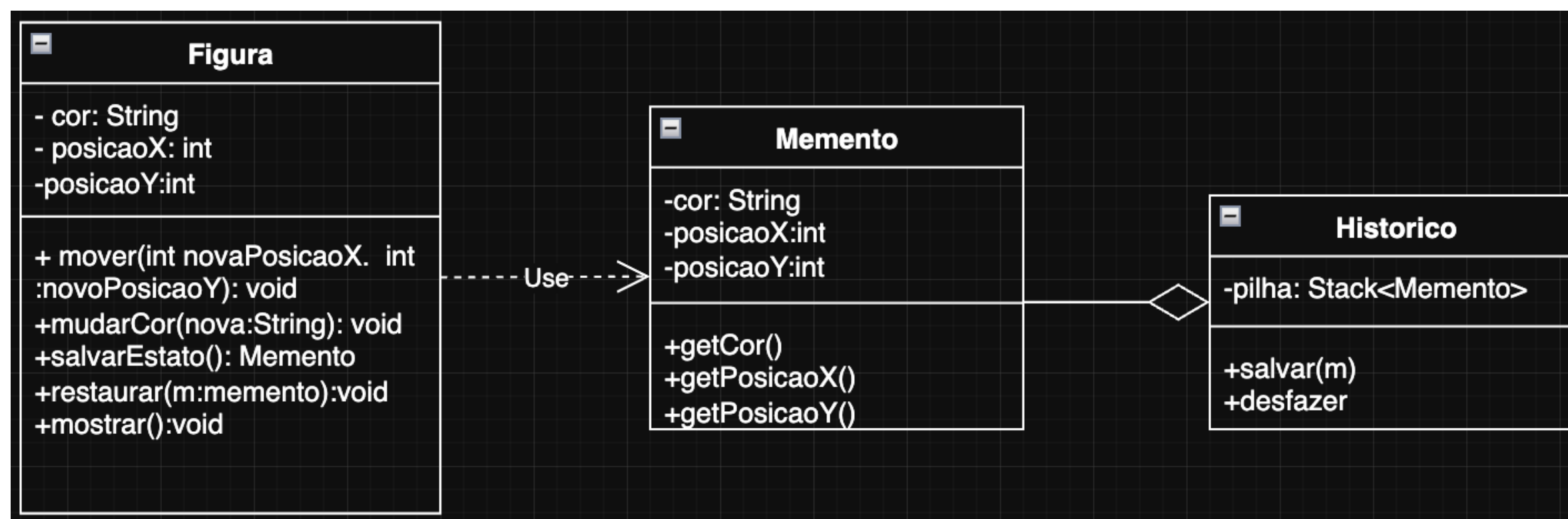


Diagrama de Classe



Implementação

A classe **Figura** salva e restaura seu próprio estado usando mementos.

```
public class Figura { // Classe Originator 2 usages  ⚡ Fernando

    private String cor; 5 usages
    private int posicaoX; 5 usages
    private int posicaoY; 5 usages

    public Figura(String cor, int posicaoX, int posicaoY) { 1 usage  ⚡ Fernando
        this.cor = cor;
        this.posicaoX = posicaoX;
        this.posicaoY = posicaoY;
    }

    public void mover(int novaPosicaoX, int novaPosicaoY) { 2 usages  ⚡ Fernando
        this.posicaoX = novaPosicaoX;
        this.posicaoY = novaPosicaoY;
    }

    public void mudarCor(String novaCor) { 2 usages  ⚡ Fernando
        this.cor = novaCor;
    }

    public Memento criarMemento() { 2 usages  ⚡ Fernando
        return new Memento(cor, posicaoX, posicaoY);
    }

    public void restaurar(Memento memento) { 2 usages  ⚡ Fernando
        this.cor = memento.getCor();
        this.posicaoX = memento.getPosicaoX();
        this.posicaoY = memento.getPosicaoY();
    }

    public void mostrar() { 5 usages  ⚡ Fernando
```

Implementação

A **classe Memento** guarda o estado da figura de forma imutável, permitindo recuperar esses valores depois sem modificá-los

```
public class Memento { 6 usages  🧑Fernando

    private final String cor; 2 usages
    private final int posicaoX; 2 usages
    private final int posicaoY; 2 usages

    public Memento(String cor, int posicaoX, int posicaoY) { 1 usage  🧑Fernando
        this.cor = cor;
        this.posicaoX = posicaoX;
        this.posicaoY = posicaoY;
    }

    public String getCor() { return cor; } 1 usage  🧑Fernando
    public int getPosicaoX() { return posicaoX; } 1 usage  🧑Fernando
    public int getPosicaoY() { return posicaoY; } 1 usage  🧑Fernando
}
```

Implementação

A **classe Historico** é responsável por armazenar e gerenciar os mementos criados pelo **Originator**, sem acessar nem modificar o conteúdo interno deles

```
import java.util.Stack;

public class Historico { 2 usages  ⚡Fernnado
    //Caretaker
    private final Stack<Memento> estados = new Stack<>(); 3 usages

    public void salvar(Memento memento) { 2 usages  ⚡Fernnado
        estados.push(memento);
    }

    public Memento desfazer() { 2 usages  ⚡Fernnado
        return estados.isEmpty() ? null : estados.pop();
    }
}
```

Implementação

Classe Main para demonstrar a criação, modificação e restauração dos estados da Figura

```
/Users/user/Library/Java/JavaVirtualMachines/corretto-  
  
Cria Figura [cor=vermelho, x=20, y=20]  
  
Modificar Figura [cor=Azul, x=200, y=200]  
  
Modificar Figura [cor=verde, x=400, y=400]  
  
Desfazer 1 Figura [cor=Azul, x=200, y=200]  
  
Desfazer 2 Figura [cor=vermelho, x=20, y=20]  
  
Nenhum estado para restaurar.  
estado atual Figura [cor=vermelho, x=20, y=20]  
  
Process finished with exit code 0
```

```
public static void main(String[] args) { @Fernando *  
  
    Historico historico = new Historico();  
    Figura figura = new Figura(cor: "vermelho", posicaoX: 20, posicaoY: 20);  
  
    System.out.println("\n Cria " + figura.mostrar());  
  
    // Salva estado inicial  
    historico.salvar(figura.criarMemento());  
  
    figura.mover(novaPosicaoX: 200, novaPosicaoY: 200);  
    figura.mudarCor(novaCor: "Azul");  
    System.out.println("\nModificar " + figura.mostrar());  
  
    // Salva segundo estado  
    historico.salvar(figura.criarMemento());  
  
    figura.mover(novaPosicaoX: 400, novaPosicaoY: 400);  
    figura.mudarCor(novaCor: "verde");  
    System.out.println("\nModificar " + figura.mostrar());  
  
    figura.restaurar(historico.desfazer());  
    System.out.println("\nDesfazer 1 " + figura.mostrar());  
  
    figura.restaurar(historico.desfazer());  
    System.out.println("\nDesfazer 2 " + figura.mostrar() + "\n");  
  
    figura.restaurar(historico.desfazer());  
    System.out.println("estado atual " + figura.mostrar());  
}
```

Consequências

✓ Vantagens

1. Protege o encapsulamento

Caretaker não sabe nada sobre o estado interno.

2. Implementa facilmente undo/redo

Estados podem ser empilhados e revertidos.

3. Simples de entender e implementar

Salvar e restaurar

4. Estados imutáveis e seguros

O memento geralmente é imutável, evitando inconsistências.

✗ Desvantagens

1. Consome muita memória

Cada memento pode ser caro dependendo do tamanho do estado.

2. Pode ter impacto na performance

Criar snapshots completos pode ser custoso.

3. Caretaker precisa de mecanismos para lidar com muitos mementos

Gerenciamento de histórico pode ser complicado.

4. Pode incentivar acoplamento oculto

Originator pode crescer demais ao lidar com múltiplos tipos de estado.

Variações

1. Memento Estrito (Clássico)

- O caretaker não conhece o estado salvo.
- Estado é completamente encapsulado.

3. Memento Empilhado (Undo/Redo)

- Duas pilhas: uma de Undo, outra de Redo.

5. Snapshot externo

- Estado serializado em arquivo (ex.: JSON).
- Variante usada para persistência.

2. Memento Transparente

- O caretaker pode acessar alguns dados internos.
- Usado em linguagens sem controle forte de visibilidade

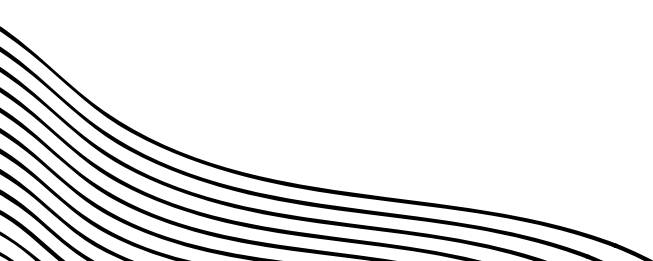
4. Memento Incremental

- Armazena só as diferenças entre estados (deltas).
- Usado quando o estado é grande (melhora memória).

Command + Undo/Redo

- Quando cada operação sabe como desfazer a si mesma.
- Quando você quer que o histórico seja baseado em ações, não em estados.
- Quando as operações são pequenas e o estado é grande.

Obs: Cada comando precisa implementar `execute()` e `undo()`, aumentando o código...



EXERCÍCIO

Criar um editor de texto simples que permita registrar e restaurar estados anteriores do texto utilizando o padrão de projeto Memento.

O exercício deve aplicar corretamente os três papéis do padrão Memento:

Originator : o editor, que possui o texto e sabe criar e restaurar estados

Memento: objeto que guarda um estado do texto

Caretaker: histórico que armazena os Mementos e permite desfazer

Referências Bibliográficas

Memento Refactoring Guru: <https://refactoring.guru/pt-br/design-patterns/memento>

Memento Teoria - Padrões de Projeto :https://www.youtube.com/watch?v=crmLq8_FtLc&t=158s

GAMMA, Erich et al. Padrões de Projeto – Soluções Reutilizáveis de Software Orientado a Objetos.